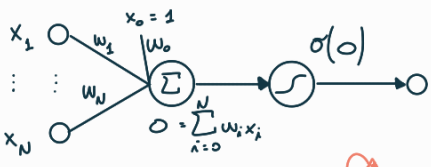
Perceptron



A single perceptron can be used to represent many boolean functions (not XOR)
A perceptron takes a vector of real-valued inputs, computes a linear combination of these inputs, applies an activation function to it and returns the result.
Learning means choosing values for the weights $w_0, \dots, w_N$

no activation

$$E(w) = \frac{1}{2} \sum_{n=1}^{N} (t_n - o_n)^2 = \frac{1}{2} \sum_{n=1}^{N} (t_n - w^T x_n)^2$$

we want to minimize $E(w)$

prediction of the model

ground truth

Contribution of the weight w_i in $E(w)$:

$$\frac{\partial E}{\partial w_i} = \frac{1}{2} \sum_{n=1}^{N} \frac{\partial}{\partial w_i} (t_n - w^T x_n)^2 = \frac{1}{\cancel{2}} \sum_{n=1}^{N} \cancel{2} (t_n - w^T x_n) \frac{\partial}{\partial w_i} (t_n - w^T x_n) = \sum_{n=1}^{N} (t_n - w^T x_n)(-x_{i,n})$$

only contribution of vector x

we can use this derivative (gradient) to implement an iterative approach to find the new value

learning rate, how much to move

$$w_i \longleftarrow w_i - \eta \frac{\partial E}{\partial w_i}$$

This procedure can fail to converge if we get stuck on a local minima
This update rule (gradient descent) provides the basis for the backpropagation (used in networks with many interconnected units)

repeat until termination condition (error < ε)

Considering all the samples can be very hard

- Batch mode ⟶ consider all the dataset D ⟶ $\Delta w_i = \eta \sum_{(x,t) \in D} (t - o(x)) x_i$
- Mini-batch mode ⟶ choose a small subset $S \subset D$ ⟶ $\Delta w_i = \eta \sum_{(x,t) \in S} (t - o(x)) x_i$
- Incremental mode ⟶ choose one sample $(x,t) \in D$ ⟶ $\Delta w_i = \eta (t - o(x)) x_i$

Termination conditions:

- predefined number of iterations
- threshold on changes in the loss function $E(w)$

Moreover the learning rate η should be sufficiently small; if we choose a large learning rate the new error value could be even bigger

N.B. moving with small η (learning rate) the solution will be very close to some samples in the dataset